Studio Nx

Backend(FastAPI/Python) 16 — verify_password()

2026-10-07

Backend(FastAPI/Python) 16 — `verify_password()`

항목	내용
파일	`studionx/backend/app/core/security.py`
작업	P1-04 회원가입 (템플릿은 P1-05 — 15 와 짝이라 함께 구현) · PRD 01 FR-103
작성	Claude (AI) · 2026-10-07 · 사용자 검토
이전 · 다음	15 `hash_password()` · 17 `normalize_email()`

1. 하는 일

입력한 비밀번호가 DB 에 저장된 해시(15)와 **맞는지** `True` / `False` 로 알려 준다.

호출	결과
`verify_password("Abcdef1!", "$2b$12$Q9xK…")` (가입 때 쓴 비밀번호)	`True`
`verify_password("Abcdef1?", "$2b$12$Q9xK…")`	`False`

해시를 원래 비밀번호로 **되돌리는 것이 아니다.** 해시 안에 있는 소금(15 §1)을 꺼내, 입력한 비밀번호를 같은 방법으로 다시 계산하고 결과가 같은지 본다.

2. 언제 쓰나

쓰는 곳	화면	하는 일
P1-05 `authenticate()`	home.html 로그인 모달	이메일로 찾은 회원의 `password_hash` 와 비교. `False` 면 401 "Email or password is incorrect."
P1-09 `check_current_password()`	userInfo.html 비밀번호 변경 · 탈퇴	현재 비밀번호 확인. `False` 면 400 "Password is incorrect."

3. 코드

import (파일 맨 위)

```
import re
import string

import bcrypt
import jwt

from app.core.config import get_settings
from app.core.errors import AppError, ErrorCode
```

`security.py` 맨 위의 import 이다 (15 와 같음, 이번에 추가한 것 없음). 이 함수는 **`bcrypt`** 만 쓴다.

함수

```
def verify_password(plain: str, hashed: str) -> bool:
    try:
        return bcrypt.checkpw(plain.encode("utf-8"), hashed.encode("ascii"))
    except ValueError:
        return False
```

문법 풀이

import 문법	뜻
`import 모듈`	모듈 전체를 가져와 `모듈.이름` 으로 씀 (예: `bcrypt.checkpw`)

식	뜻
`bcrypt.checkpw(비밀번호, 해시)`	해시 안의 소금으로 비밀번호를 다시 계산해 같으면 `True`. 둘 다 바이트로 넘김
`plain.encode("utf-8")` · `hashed.encode("ascii")`	글자 → 바이트 (15 의 반대 방향)
`try: … except ValueError: return False`	`ValueError` 가 나면 오류로 멈추지 않고 `False`

입력	`bcrypt.checkpw`	이 함수
맞는 비밀번호	`True`	`True`
틀린 비밀번호	`False`	`False`
72바이트가 넘는 비밀번호 (`"A" * 100`)	**`ValueError`**	`False`
망가진 해시 (`""`, `"x"`)	**`ValueError`** ("Invalid salt")	`False`

4. 흐름

1. 비밀번호와 해시를 바이트로 바꾼다.
2. bcrypt 가 해시에서 소금 · 비용을 꺼내 비밀번호를 다시 계산한다 (약 0.2초).
3. 결과가 같으면 `True`, 다르면 `False`.
4. 계산할 수 없는 입력이면 `False`.

5. 왜 이렇게

선택	이유
`ValueError` 를 `False` 로	로그인 입력은 100자까지 받는다(`LoginRequest`). 비밀번호 규칙(14)은 가입 때만 검사하므로, 로그인 칸에 긴 글을 넣으면 bcrypt 가 오류를 던져 **500** 이 난다. 그런 비밀번호로 가입한 회원은 없으니 "틀림"이 맞는 답이다
망가진 해시도 `False`	DB 에 잘못된 값이 있어도 로그인이 500 으로 깨지지 않고 "틀림"으로 끝난다
`==` 로 직접 비교하지 않음	`bcrypt.checkpw` 는 비교에 걸리는 시간이 결과와 상관없이 같다. 시간 차이로 해시를 추측하는 공격을 막는다
템플릿(P1-05)보다 먼저 구현	15 와 짝이라, 15 의 시험을 "해시 → 비교" 로 함께 할 수 있다

P1-05 에서 알아 둘 것: 이메일이 없는 회원이면 이 함수를 부르지 않아 응답이 0.2초 빨라진다. 그 차이로 "이 이메일은 가입되어 있다"는 것을 알아낼 수 있다. 그래서 `authenticate()` 템플릿에는 회원이 없을 때도 가짜 해시로 이 함수를 한 번 부르라고 적혀 있다.

6. 시험 (`tests/test_password.py` 에 11개 추가 · 모두 32개)

시험	확인한 것
맞는 비밀번호	`True`
틀린 비밀번호 5개	한 글자 다름 · 대소문자 · 뒤 공백 · 빈 글자 · 두 번 반복 → `False`
너무 긴 비밀번호	`"A" * 100` · 한글 30자(90바이트) → 오류 없이 `False`
망가진 해시 4개	`""` · `"x"` · `"not-a-bcrypt-hash"` · `"$2b$12$짧음"` → 오류 없이 `False`

전체 시험 108개 통과, `ruff` 검사 통과.